

JDBC - Cheat Sheet

Contents

1. [How JDBC fits together](#)
2. [Driver types \(interview classic\)](#)
3. [Connecting to the database](#)
4. [DriverManager vs DataSource](#)
5. [Statement vs PreparedStatement vs CallableStatement](#)
6. [ResultSet](#)
7. [Transactions](#)
8. [Batch updates](#)
9. [Auto-generated keys](#)
10. [Exception handling](#)
11. [Resource handling](#)
12. [Large result sets](#)
13. [RowSet \(rare\)](#)
14. [JDBC vs higher-level tools](#)
15. [Best practices](#)
16. [Common gotchas](#)
17. [Quick reference](#)

JDBC (Java Database Connectivity) is the standard Java API for talking to relational databases. It's been part of the JDK since 1997 and remains the foundation every higher-level data tool (Hibernate, JPA, Spring JDBC, MyBatis) sits on top of.

You write SQL, JDBC ships it to the database, and you read back rows. The API is verbose but predictable.

How JDBC fits together

Four core interfaces handle every interaction:

Interface	Job
<code>Connection</code>	A live session with the database
<code>Statement</code>	A SQL command without parameters
<code>PreparedStatement</code>	A precompiled SQL command with bind parameters
<code>ResultSet</code>	The rows that came back from a query

Plus `DriverManager` (the older way to get a connection) and `DataSource` (the modern way, used by connection pools).

Driver types (interview classic)

JDBC ships drivers in 4 types. Almost all production code uses Type 4.

Type	Name	How it works
Type 1	JDBC-ODBC bridge	Translates JDBC -> ODBC. Removed in Java 8.
Type 2	Native API	JDBC -> native C library. Platform-specific.
Type 3	Network protocol	JDBC client -> middleware server -> DB. Rare.
Type 4	Thin (pure Java)	JDBC directly speaks the DB's wire protocol. Default for Postgres, MySQL, Oracle, SQL Server.

Type 4 drivers are pure Java, portable across platforms, and don't need extra middleware. That's what `postgresql-jdbc`, `mysql-connector-j`, and similar ship.

Connecting to the database

```
String url = "jdbc:postgresql://localhost:5432/mydb";
String user = "alice";
String pass = "secret";

try (Connection conn = DriverManager.getConnection(url, user, pass)) {
    // use conn
}
```

URL format depends on the DB:

```
jdbc:postgresql://host:5432/dbname
jdbc:mysql://host:3306/dbname
jdbc:sqlserver://host:1433;databaseName=dbname
jdbc:oracle:thin:@host:1521:sid
```

`Class.forName("org.postgresql.Driver")` is no longer needed in modern JDBC (since JDBC 4.0 / Java 6). The driver auto-registers via the ServiceLoader mechanism.

DriverManager vs DataSource

`DriverManager.getConnection(...)` opens a new TCP connection every call. Expensive. Use it only for quick scripts or tests.

`DataSource` is the modern interface. Production code gets connections from a pool:

```
HikariConfig config = new HikariConfig();
config.setJdbcUrl("jdbc:postgresql://localhost:5432/mydb");
config.setUsername("alice");
config.setPassword("secret");
config.setMaximumPoolSize(20);

HikariDataSource ds = new HikariDataSource(config);

try (Connection conn = ds.getConnection()) {
    // pool returns a wrapped connection. close() returns it to the pool.
}
```

HikariCP is the default in Spring Boot. Faster than DBCP and C3P0.

Statement vs PreparedStatement vs CallableStatement

Three flavors. Pick `PreparedStatement` 99% of the time.

Statement

For SQL with no parameters. Reparses on every execution.

```
try (Statement stmt = conn.createStatement();
     ResultSet rs = stmt.executeQuery("SELECT COUNT(*) FROM users")) {
    rs.next();
    int count = rs.getInt(1);
}
```

Vulnerable to SQL injection if you string-concatenate user input. Don't.

PreparedStatement

For SQL with bind parameters. The database parses the SQL once and reuses the plan. Also escapes parameters safely.

```
String sql = "SELECT id, email FROM users WHERE status = ? AND created_at > ?";

try (PreparedStatement ps = conn.prepareStatement(sql)) {
    ps.setString(1, "active");
    ps.setTimestamp(2, Timestamp.from(Instant.now().minus(7, ChronoUnit.DAYS)));

    try (ResultSet rs = ps.executeQuery()) {
        while (rs.next()) {
            System.out.println(rs.getLong("id") + " " + rs.getString("email"));
        }
    }
}
```

Bind parameters are 1-indexed. Type-specific setters: `setString`, `setLong`, `setTimestamp`, `setObject`, `setNull`, etc.

CallableStatement

For stored procedures. Same pattern as `PreparedStatement` plus OUT parameter registration.

```
String sql = "{ call get_user_summary(?, ?) }";

try (CallableStatement cs = conn.prepareCall(sql)) {
    cs.setLong(1, 42);
    cs.registerOutParameter(2, Types.INTEGER);
    cs.execute();
    int summary = cs.getInt(2);
}
```

Use it only when the team relies on stored procs.

ResultSet

The cursor over query results. Forward-only by default. Call `next()` to advance.

```
try (PreparedStatement ps = conn.prepareStatement("SELECT id, name, age FROM users");
    ResultSet rs = ps.executeQuery()) {

    while (rs.next()) {
        long id = rs.getLong("id");
        String name = rs.getString("name");
        int age = rs.getInt("age");

        // age will be 0 if the column was NULL. Check rs.wasNull() if that matters.
    }
}
```

Getter naming follows the column type:

```
rs.getLong("id");
rs.getString("name");
rs.getBigDecimal("price");
rs.getTimestamp("created_at");
rs.getBoolean("is_active");
rs.getObject("metadata", Map.class); // works for JSON columns with the right driver
```

NULL handling: primitive getters return 0 or false for SQL NULL. To detect NULL, call `rs.isNull()` right after the getter, or use the boxed getter with `getObject`.

Transactions

By default, every JDBC statement auto-commits. Turn that off for multi-step operations.

```
try (Connection conn = ds.getConnection()) {
    conn.setAutoCommit(false);

    try (PreparedStatement debit = conn.prepareStatement(
        "UPDATE accounts SET balance = balance - ? WHERE id = ?");
        PreparedStatement credit = conn.prepareStatement(
            "UPDATE accounts SET balance = balance + ? WHERE id = ?")) {

        debit.setBigDecimal(1, amount);
        debit.setLong(2, fromId);
        debit.executeUpdate();

        credit.setBigDecimal(1, amount);
        credit.setLong(2, toId);
        credit.executeUpdate();

        conn.commit();
    } catch (SQLException e) {
        conn.rollback();
        throw e;
    } finally {
        conn.setAutoCommit(true); // restore default before returning to pool
    }
}
```

Savepoints for partial rollback:

```
Savepoint sp = conn.setSavepoint("after-debit");
try {
    credit.executeUpdate();
} catch (SQLException e) {
    conn.rollback(sp); // roll back to the savepoint, keep the debit
}
```

Isolation levels:

```
conn.setTransactionIsolation(Connection.TRANSACTION_READ_COMMITTED);
```

Constants: `TRANSACTION_READ_UNCOMMITTED`, `TRANSACTION_READ_COMMITTED`,
`TRANSACTION_REPEATABLE_READ`, `TRANSACTION_SERIALIZABLE`.

Batch updates

For bulk inserts or updates, batching beats one-at-a-time round trips.

```
String sql = "INSERT INTO events (user_id, type, payload) VALUES (?, ?, ?)";

try (PreparedStatement ps = conn.prepareStatement(sql)) {
    for (Event e : events) {
        ps.setLong(1, e.userId());
        ps.setString(2, e.type());
        ps.setString(3, e.payload());
        ps.addBatch();
    }
    int[] counts = ps.executeBatch();
}
```

Postgres needs `rewriteBatchedInserts=true` in the JDBC URL to actually batch on the wire. Otherwise it still sends one INSERT per row.

```
jdbc:postgresql://host/db?rewriteBatchedInserts=true
```

Auto-generated keys

When the DB generates the primary key (e.g., `SERIAL`, `AUTO_INCREMENT`), retrieve it after insert.

```
String sql = "INSERT INTO users (name, email) VALUES (?, ?)";

try (PreparedStatement ps = conn.prepareStatement(sql,
    Statement.RETURN_GENERATED_KEYS)) {
    ps.setString(1, "alice");
    ps.setString(2, "alice@example.com");
    ps.executeUpdate();

    try (ResultSet keys = ps.getGeneratedKeys()) {
        if (keys.next()) {
            long id = keys.getLong(1);
        }
    }
}
```

Exception handling

`SQLException` carries three useful bits:

- `getMessage()` : human-readable error.
- `getSQLState()` : the standard 5-character ANSI code (e.g., 23505 for unique violation).
- `getErrorCode()` : the vendor's numeric code.

Prefer `SQLState` for portable error handling. Vendor codes only when you have to.

```
try {
    ps.executeUpdate();
} catch (SQLException e) {
    if ("23505".equals(e.getSQLState())) {
        throw new DuplicateEmailException();
    }
    throw e;
}
```

`SQLException` is a checked exception. It chains via `getNextException()`. Use `e.iterator()` or walk the chain manually to log all causes.

Resource handling

Every JDBC object holds a real DB resource (connection, cursor, prepared statement handle). Leaks lead to connection pool exhaustion and the DB rejecting new sessions.

Use try-with-resources for everything that implements `AutoCloseable` :

```
try (Connection conn = ds.getConnection();
     PreparedStatement ps = conn.prepareStatement(sql);
     ResultSet rs = ps.executeQuery()) {
    // ...
}
```

All three are auto-closed in reverse order when the block exits. For pre-Java 7 code or where try-with-resources doesn't fit, close in `finally`. Always.

Large result sets

By default, JDBC drivers buffer the whole result set in memory. For a million-row query, that's a problem.

Postgres uses cursors when `autoCommit` is off and you set a fetch size:

```

conn.setAutoCommit(false);
try (PreparedStatement ps = conn.prepareStatement("SELECT * FROM events")) {
    ps.setFetchSize(1000);

    try (ResultSet rs = ps.executeQuery()) {
        while (rs.next()) {
            // processed in 1000-row chunks
        }
    }
}
}

```

MySQL has its own quirk: `Statement.setFetchSize(Integer.MIN_VALUE)` enables streaming, with caveats. Read the driver docs before relying on it.

RowSet (rare)

`RowSet` extends `ResultSet`. Variants: `CachedRowSet` (disconnected, in-memory), `JdbcRowSet` (wraps an open connection), `WebRowSet` (XML serializable). Almost never seen in modern code. Mention exists. Details live in the JDK docs.

JDBC vs higher-level tools

Layer	Tool	When to pick
Raw JDBC	What's here	Maximum control, simple data access, scripts
Spring JDBC / JdbcTemplate	Wraps boilerplate	When you want SQL but not the verbosity
jOOQ	Type-safe SQL DSL	Complex SQL, code-generated schema
MyBatis	XML or annotation SQL maps	When SQL lives in config
Hibernate / JPA	ORM	Mapping objects to tables, change tracking

JPA / Hibernate is the most common in Spring Boot apps but adds complexity (N+1 problems, `LazyInitializationException`, dirty-checking quirks). Raw JDBC or JdbcTemplate is sometimes the right call for hot paths.

Best practices

- Always use `PreparedStatement` for queries that take parameters. Skip `Statement` unless you really have no parameters.
- Always use try-with-resources. Every leaked connection is a future pager.

- Always get connections from a `DataSource` (HikariCP) in production. `DriverManager` belongs in scripts.
- Set explicit isolation levels for transactions where it matters.
- Set a fetch size for large queries to avoid loading everything into memory.
- Batch inserts and updates when you have more than 10 rows to write.
- Restore `autoCommit` to true before returning a connection to the pool.
- Map `SQLState` codes to domain exceptions in one place.
- Log SQL and parameters at `DEBUG`, never `INFO` (parameters can contain PII).
- Don't open new connections inside loops. Reuse the one connection across an operation.

Common gotchas

- Forgetting `conn.commit()` with auto-commit off. Your INSERT looks fine in the app, then disappears when the connection returns to the pool and rolls back.
- Returning a connection to the pool with auto-commit off. Next caller gets a transactional connection without knowing it. Reset it in `finally`.
- String concatenation with user input. Classic SQL injection. Use parameter binding.
- Iterating a `ResultSet` after closing the `PreparedStatement`. The cursor closes with the statement. Read everything you need first.
- Not handling `wasNull()` on primitive getters. `getInt("optional_count")` returns 0 for both 0 and NULL. Use boxed getters or `wasNull()` if the difference matters.
- Connection per request without a pool. Each request pays the TCP and auth handshake. Use HikariCP.
- Connection pool too large. A pool of 200 connections fighting over 10 cores is slower than 10 connections. Tune based on actual concurrency.
- Batching without driver support. `addBatch()` only helps when the driver actually batches on the wire. Check the driver-specific flag.
- Loading the whole result set for large queries. Set `fetchSize` and turn off auto-commit (Postgres).
- Swallowing `SQLException` chains. Walk `getNextException()` to surface the real cause.
- `PreparedStatement` cached too aggressively in some pools. If the schema changes, old plans fail. Rotate or invalidate the pool.

Quick reference

Operation	Code
Get connection	<code>ds.getConnection()</code>
Prepare query	<code>conn.prepareStatement(sql)</code>
Bind parameter	<code>ps.setString(1, "value")</code>
Run query	<code>ps.executeQuery()</code>

Operation	Code
Run update	<code>ps.executeUpdate()</code>
Batch update	<code>ps.addBatch()</code> + <code>ps.executeBatch()</code>
Read row	<code>rs.next()</code> + <code>rs.getX("col")</code>
Auto-commit off	<code>conn.setAutoCommit(false)</code>
Commit	<code>conn.commit()</code>
Rollback	<code>conn.rollback()</code>
Savepoint	<code>conn.setSavepoint("name")</code>
Get generated key	<code>ps.getGeneratedKeys()</code>
Pool	<code>HikariDataSource</code>
Detect unique violation	<code>e.getSQLState().equals("23505")</code>
Stream large result	<code>ps.setFetchSize(1000)</code> with <code>autoCommit=false</code>